

Conversación con Gemini

¿Qué herramienta me recomendás para leer y modificar archivos Excel desde Python? ¿Por qué sería una buena opción para este proyecto?

Para este proyecto, la librería de Python que te recomiendo de forma directa es **openpyxl**.

Si bien existen opciones muy potentes en el ecosistema de Python, las características específicas de tu Trabajo Práctico Integrador (TPI) hacen que esta sea la herramienta ideal.

¿Por qué **openpyxl** es la mejor opción para este proyecto?

1. Ideal para simular la persistencia de datos (Base de Datos)

Las consignas de tu trabajo especifican que el uso de una base de datos es obligatorio, permitiendo explícitamente que esta sea **una plantilla de Excel con las posibles respuestas o registros**. **openpyxl** te permite leer celdas específicas de manera muy sencilla para simular estas consultas en tiempo real (por ejemplo, validar si un usuario tiene días disponibles o si un presupuesto está aprobado).

PDF+ 3

2. Soporte nativo para lectura y escritura dinámicas

Un chatbot administrativo profesional no puede basarse en respuestas estáticas; necesita guardar estados y registrar información. Con **openpyxl** podés hacer ambas cosas:

PDF+ 1

Lectura: Buscar datos ingresados por el usuario para validar reglas de negocio.

PDF

Escritura: Registrar nuevas solicitudes (como la carga de un proveedor o un pedido de vacaciones) de punta a punta en el archivo simulado.

PDF+ 1

3. Es liviana y no requiere dependencias externas complejas

A diferencia de otras librerías masivas como `pandas` (que está diseñada para el análisis de datos masivos y Big Data), `openpyxl` está optimizada exclusivamente para interactuar con archivos `.xlsx`. Esto mantiene el código de tu chatbot más limpio, enfocado y fácil de estructurar en tu repositorio de GitHub.

PDF

Un ejemplo rápido de cómo se vería en tu código

Para que veas lo amigable que es, te dejo una estructura básica de cómo podrías usarla para simular que tu bot consulta el "saldo de vacaciones" de un empleado:

PDF

```
Python
import

def consultar_saldo_vacaciones
    # Cargamos el archivo Excel que actúa como base de
    datos

    'base_datos_simulada.xlsx'

    # Recorremos las filas buscando al empleado
    for      in      2
        True
        if      0      # Suponiendo que la
columna 1 es el ID
        return      2      # Suponiendo que la
columna 3 son los días disponibles

    return 0 # Si no lo encuentra
```

Otras alternativas (y por qué no las elegiría en este caso)

`pandas`: Es la librería reina para Ciencia de Datos. Aunque puede leer y escribir Excel, añade mucha complejidad innecesaria para la

estructura lógica (estructuras `$if/else$` o `$switch/case$`) que requiere la máquina de estados de tu bot.

PDF+ 1

`xlrd` / `xlwt`. Son librerías antiguas. Hoy en día ya no dan soporte nativo para los formatos modernos de Excel (`.xlsx`), quedando limitadas a versiones obsoletas.

`openpyxl` te dará el equilibrio justo entre facilidad de uso y la robustez necesaria para cubrir tanto el "camino feliz" como el manejo de excepciones de tu simulación.

PDF+ 2

¿Qué programa puedo utilizar para crear diagramas BPMN de forma sencilla y profesional? ¿Existen herramientas gratuitas recomendadas?

Para crear diagramas **BPMN 2.0** de forma sencilla, profesional y que además se adapten perfectamente a los requisitos de tu trabajo práctico (donde necesitás exportar en alta resolución y definir carriles, compuertas y eventos), existen excelentes opciones gratuitas.

Aquí tenés las herramientas más recomendadas, divididas según su enfoque:

1. La opción ideal para Programadores: Camunda Modeler

Es una de las herramientas más respetadas en el mundo del desarrollo de software porque está diseñada específicamente para BPMN 2.0.

¿Por qué elegirla? Es **100% gratuita** y de código abierto. Al estar pensada para desarrolladores, la estructura visual que generás coincide de manera exacta con la lógica que después vas a plasmar en tu código (las tareas de usuario, de servicio y las compuertas).

Formatos: Te permite modelar en su versión de escritorio (Desktop) o en la nube, y exportar diagramas en imágenes de alta resolución (PNG, SVG) o directamente en archivos `.bpmn` (XML), lo cual es ideal si en el futuro querés integrar el diagrama con un motor de procesos.

Precio: Gratis.

2. La más intuitiva y colaborativa: Bizagi Modeler

Bizagi es un estándar de la industria y es famosa por su facilidad de uso mediante el sistema de "arrastrar y soltar".

¿**Por qué elegirla?** Su interfaz es sumamente limpia y profesional. Si hacés el trabajo en grupo (hasta dos integrantes), Bizagi facilita mucho el diseño en equipo. Te ayuda de forma automática a cumplir con las reglas de la notación BPMN, avisándote si estás conectando mal una compuerta o un evento.

Formatos: Exporta en PDF de alta calidad, PNG y Word (te genera documentación automática de los procesos, lo que te puede ahorrar tiempo para el informe).

Precio: Tiene un **plan gratuito muy completo** para uso personal/académico.

3. La opción 100% web y rápida: Miro o Lucidchart

Si no querés instalar nada y buscás algo muy visual para empezar a estructurar el "As-Is" y el "To-Be" rápidamente.

¿**Por qué elegirla?** **Lucidchart** tiene plantillas nativas específicas para BPMN 2.0 con todos los íconos oficiales (carriles, eventos de inicio/fin, compuertas). **Miro**, por su parte, es un lienzo infinito espectacular para una lluvia de ideas inicial con tu compañero de equipo.

Precio: Ambos tienen **versiones gratuitas**. Lucidchart te limita a 3 documentos y un número máximo de figuras por diagrama en su cuenta gratis, pero para el alcance de este TPI suele ser suficiente.

Resumen de recomendación para tu TPI:

📌 **Mi sugerencia:** Si querés una herramienta que te obligue a mantener la **coherencia estructural** entre el negocio y el código de tu bot, bajate **Camunda Modeler**. Si preferís priorizar la rapidez de diseño y una estética impecable para el PDF final de forma muy simple, usá **Bizagi** o **Lucidchart**. ¿Ya tenés definido cuál va a ser el proceso administrativo crítico (vacaciones, soporte, proveedores) que vas a modelar para tu chatbot?

Estoy usando Visual Studio Code. ¿Qué extensiones pueden ayudarme a trabajar con archivos Python, diagramas y archivos Excel durante el desarrollo del proyecto?

Trabajar con todo centralizado en Visual Studio Code te va a ahorrar mucho tiempo de desarrollo y evitará que estés saltando de ventana en ventana.

Para armar el entorno ideal para tu proyecto, estas son las extensiones indispensables divididas por cada necesidad:

1. Para la programación del Bot (Python)

Python (de Microsoft): Es la extensión oficial y obligatoria. Te ofrece autocompletado inteligente (IntelliSense), formateo automático de código para mantenerlo limpio y la capacidad de ejecutar y depurar tu código fácilmente.

Python Debugger (de Microsoft): Te va a salvar la vida cuando programes la **máquina de estados** de tu bot. Te permite pausar la ejecución del código en una línea específica (breakpoints) para inspeccionar las variables en tiempo real y ver si tu flujo está tomando el camino lógico correcto o el "camino infeliz".

2. Para visualizar y editar tu "Base de Datos" (Excel)

XLSX, CSV, TSV & Markdown Editor (de Muhammad Ahmad):

Excelente opción para este proyecto. Te permite abrir tus archivos **.xlsx** simulados directamente dentro de VS Code en una pestaña con formato de cuadrícula visual (con colores y estilos). Podés editar valores en las celdas de forma interactiva y guardar los cambios, ideal para preparar las filas de prueba (como los saldos de vacaciones o datos de proveedores).

Excel Viewer (de GrapeCity): Si solo necesitás visualizar la tabla de forma rápida y limpia para verificar que tu script de Python esté insertando o leyendo los datos correctamente, esta alternativa es sumamente ligera y eficiente.

3. Para diseñar el modelo de negocio dentro de VS Code (BPMN)

BPMN Modeler (de Miragon GmbH): Esta extensión integra el motor visual de bpmn.io (el mismo que usa Camunda) directo en tu editor.

¿**Por qué te sirve?** Te permite crear, ver y editar archivos **.bpmn** sin salir de VS Code. Podés arrastrar compuertas, eventos y carriles (lanes) para diseñar tus flujos "As-Is" y "To-Be".

Tip para la entrega: Tiene comandos directos para **exportar tu diagrama como archivo SVG o PNG**, cumpliendo fácilmente con el requisito de entregar una imagen de alta resolución en tu informe PDF.

4. Para la documentación y control de versiones

GitLens — Git supercharged: Como la cátedra te exige el uso obligatorio de un repositorio en GitHub, GitLens te ayuda a visualizar de forma súper clara el historial de cambios en tu código línea por línea.

Markdown All in One: Te va a facilitar muchísimo la escritura del archivo `README.md` que te piden para la defensa final, dándote atajos de teclado para negritas, listas y previsualizaciones del texto en tiempo real.



Consejo de flujo de trabajo:

Podés dividir la pantalla de VS Code en tres columnas: en la izquierda ponés tu código de Python `bot.py`, en el centro el diagrama interactivo de `BPMN Modeler` como "contrato" de tu lógica, y en la derecha la "base de datos" simulada en Excel abierta con el editor de XLSX. ¡Así vas a tener un centro de control completísimo para simular el proceso de punta a punta!

Necesito preparar el proyecto para subirlo a GitHub. ¿Cómo debería armar el archivo `.gitignore`, el `requirements.txt` y la estructura general del repositorio?

Para que tu repositorio de GitHub se vea súper profesional, ordenado y cumpla con todo lo solicitado por la cátedra (incluyendo el código limpio y la documentación para la defensa), es clave estructurarlo correctamente desde el primer día.

A continuación, te detallo cómo armar la estructura de carpetas, el archivo `.gitignore` y el `requirements.txt`.

1. Estructura general del repositorio

Te recomiendo separar la lógica del negocio (el diagrama), la documentación y el código fuente. Una estructura ideal para este proyecto de la UTN sería la siguiente:

Plaintext

2. El archivo `.gitignore`

El archivo `.gitignore` le dice a Git qué archivos **no** debe subir a GitHub (archivos temporales, entornos virtuales, configuraciones locales). Creá un archivo llamado `.gitignore` (con el punto al principio) en la raíz del proyecto con este contenido:

```
Ini, TOML
```

```
# --- Entorno Virtual de Python ---
```

```
# --- Archivos temporales de Python ---
```

```
# --- Configuraciones del Editor (VS Code) ---
```

```
# --- Archivos del Sistema Operativo ---
```

```
# --- IMPORTANTE: Archivos temporales de Excel ---
```

```
# Evita que se suban los archivos de bloqueo creados por  
Excel mientras editás.
```



Nota importante para tu TPI: Normalmente, los archivos de datos (`.xlsx` o `.db`) se ignoran en proyectos reales. Sin embargo, como en este trabajo práctico el archivo Excel actúa como tu base de datos simulada y obligatoria, **sí debes permitir que Git lo rastree** para que los profesores puedan clonar tu repositorio y probar el bot con tus datos de ejemplo. Por eso solo ignoramos los temporales `~$*.xlsx`

3. El archivo `requirements.txt`

Este archivo lista las librerías externas que los docentes o tu compañero necesitarán instalar para que el proyecto funcione. Basándome en lo que charlamos antes, tu archivo básico debería contener:

Plaintext

Para generar este archivo automáticamente desde tu consola una vez que tengas todo instalado, podés ejecutar el comando: `pip freeze > requirements.txt`



Un último tip para el README.md

La cátedra hace mucho hincapié en que actúes como un **consultor tecnológico**. Asegurate de que el `README.md` no sea solo código; agregá una sección inicial atractiva que explique qué proceso administrativo estás resolviendo, por qué era ineficiente y cómo tu bot automatiza ese flujo de punta a punta. ¡Eso te va a sumar muchísimos puntos en la defensa final!

Gemini es una IA y puede cometer errores.